

Systems Analysis Study Guide

Lecture Notes - Visual Summary

RAD vs JAD (Lecture 2)

JAD Joint Application Development



Fact-Finding Technique

Team-based requirements gathering

- Brings users into development as **active participants**
- More expensive, can be cumbersome with large groups
- Results in **accurate requirements** statement
- Stronger commitment to system success

Focus: One phase (requirements modeling)

RAD Rapid Application Development



Complete Process

Prototyping & user involvement

- Users examine **working model early**
- Cuts development time & expense
- Limits cost of changes in long schedules
- Great for complex user interfaces

⚠ Weakness: May not emphasize strategic business needs



Key Difference: JAD focuses on **team-based fact-finding** (one phase), while RAD is a **compressed entire process**

Agile

Key Values



Individuals & Interactions

over ~~Processes & Tools~~



Working Software

over ~~Comprehensive Documentation~~



Customer Collaboration

over ~~Contract Negotiation~~



Responding to Change

over ~~Following a Plan~~

Agile = 4 Values + 12 Principles

Scrum Teams

Teams are **self-organized** and **self-directed**. Unlike other approaches, Scrum teams do not have a designated team leader.

Daily Stand-up Meeting (SCRUM)

1

What I did yesterday?

2

What I'll do today?

3

Any impediments I'm facing?

Scrum Framework Deep Dive

Who Manages the Scrum Team?

Nobody.

Scrum teams are **self-organizing**

Scrum Roles Deep Dive

Product Owner

"Advocates for customer, manages the product backlog, and helps prioritize work done by the development team."

Key Responsibilities:

- Talk to stakeholders/users to understand the product vision and their challenges
- Helps the team understand the needs and challenges to solve
- Writes User Stories (simple feature descriptions from end-user perspective)
- Responsible for managing the Product Backlog

Product Backlog Management includes:

1. Clearly expressing Product Backlog items
2. Ordering items to best achieve goals and missions
3. Optimizing the value of work the Development Team performs
4. Ensuring the Product Backlog is visible, transparent, and clear to all
5. Ensuring the Development Team understands items to the level needed

Scrum Master

"Responsible for ensuring Scrum is understood and enacted. Ensures the Scrum Team adheres to Scrum theory, practices, and rules."

Key Responsibilities:

- Finding techniques for effective Product Backlog management
- Ensuring the Product Owner knows how to arrange the Product Backlog to maximize value
- Understanding and practicing agility
- Facilitating Scrum events as requested
- Removing impediments for the team

</> Development Team

"Professionals who deliver a potentially releasable Increment of 'Done' product at the end of each Sprint."

Key Characteristics:

- ✓ Self-organizing: No one (not even the Scrum Master) tells them how to turn Product Backlog into Increments
- ✓ Cross-functional: Has all skills necessary to create a product Increment as a team
- ✓ No sub-teams: Regardless of particular domains (testing, business analysis, etc.) - no exceptions to this rule
- ✓ Collective accountability for delivering increments

📊 Key Metrics & Artifacts



Velocity

Story points completed per sprint (central metric)



Burndown Chart

Tracks velocity & remaining work



Product Backlog

Prioritized list of value to deliver



Increment

The product of a Sprint

Product Backlog

- Prioritized list of value the team can deliver
- Product Owner owns & maintains it
- Can be added, changed, reprioritized

Task Board

- Lists backlog items team is working on
- Broken down into tasks
- Shows progress (To Do → Done)

Sprint Ceremonies

1 Sprint Planning

2 Daily Scrum

Team chooses backlog items for upcoming sprint

15-min standup to sync progress

3

Sprint Review

Team demonstrates accomplishments to stakeholders

4

Sprint Retrospective

Reflect on what went well & areas to improve

Backlog = A set of work items ordered and stated (requirements, bugs to solve)

Comparison

Scrum vs Kanban

Aspect	Scrum	Kanban
Work Flow	Fixed length sprints	Continuous flow model
Roles	Defined roles (PO, SM, Dev)	No specific roles defined
Key Metric	Velocity (story points/sprint)	Cycle time



When to Use Scrum

When IT team supports **many organizations** (like IT company)



When to Use Kanban

When IT team supports **one organization**



Scrum is Iterative

Agile Method: Pros & Cons

✓ Advantages

- + Very flexible and efficient in dealing with change
- + Frequent deliverables constantly validate the project and reduce risk
- + Stress team interaction and reflect community-based values

× Disadvantages

- Team members need high level of technical and interpersonal skills
- Lack of structure and documentation can introduce risk factors
- Blurring of roles/responsibilities, loss of corporate knowledge

Lecture 3

Factors & Feasibility

External Factors



Technology



Government



Suppliers



Economy



Competitors

Internal Factors

Organizational structure, resources, culture, and strategic goals

Feasibility Study Types



Operational

Review of user needs, requirements, and expectations



Technical

Hardware, software, and network resources needed



Economic

Cost-benefit analysis and ROI



Schedule

Timeline feasibility assessment

Discretionary Projects

Management has a **choice** in implementing them

Non-discretionary Projects

No choice exists - must be done

Analysis Tools

PICK Chart: Find requests with low effort, high feasibility, and importance

Pareto Chart: Maps issues by descending frequency with cumulative curve



Project Manager

Responsible for overseeing all relevant tasks

Planning

Scheduling

Monitoring & Controlling

Reporting

Project Scheduling Tools



Gantt Chart

Horizontal bar chart with **tasks on vertical axis** and **time intervals on horizontal axis**



PERT/CPM

Bottom-up technique - analyzes complex projects as series of individual tasks

Work Breakdown Structure (WBS):

Breaking project into smaller tasks

Network Diagram:

Complete chain of task patterns for a project



Risk Management

Process of identifying, analyzing, anticipating, and monitoring risks

Brooks' Law: Adding more people to a project might actually **increase completion time**

System Analysis

⚠ It may be tempting to "just do something" but a systems project that **does not satisfy business requirements serves no useful purpose**

Requirements by Methodology



Structured Development → [Software Requirements Specification \(SRS\)](#)



Object Oriented → [Use Case Diagram](#)



Agile Development → [Personas & User Stories](#)

Systems Analyst Skills



Analytical Skills

Identify problems, evaluate key elements, develop useful solutions



Interpersonal Skills

Work with people at all organizational levels

Modeling Notations & Frameworks

Key Modeling Notations

FDD

Functional Decomposition Diagram

Top-down representation of a function or process

BPMN

Business Process Modeling Notation

Standard language used during requirements modeling

UML

Unified Modeling Language

Widely used method of visualizing and documenting software systems design

Enterprise Architecture Frameworks

Zachman Framework

- More structured approach
- Popular for modeling enterprise architectures
- Matrix-based (What, How, Where, Who, When, Why)

TOGAF Framework

- Aligns IT goals with business goals
- Organizes cross-departmental IT efforts
- Defines & organizes requirements before project starts
- Created by The Open Group

Key Difference: Zachman = Architecture Matrix | TOGAF = **Process & Methodology**

Fact-Finding Techniques

Q During Requirements Modeling

Interviews

Document Review

Observation

Surveys & Questionnaires

Sampling

Research

👤 Interview Steps

- 1 Determine People to Interview
- 2 Establish Objectives
- 3 Develop Questions
- 4 Prepare for Interview
- 5 Conduct Interview
- 6 Document Interview
- 7 Evaluate Interview

👥 Group Interviews

Can save time & observe interaction, but can also present problems

👁️ Personal Observation

Verify interview statements, see if procedures operate as described

⚠️ Hawthorne Effect

People may behave differently when they know they're being observed. Consider this factor during personal observation.

Informal Structures

Usually based on **interpersonal relationships**

📋 Questionnaires

Lec 8

Agile Modeling & Personas

Agile Modeling (AM)

A **practice-based methodology** for effective modeling and documentation of software-based systems. At a high level, AM is a collection of Core Practices.



What are Personas?

VERY IMPORTANT!

Fictional characters created based on research to represent different user types that might use your service, product, site, or brand in a similar way.



Role-Based Persona

- Goal-directed and focuses on behavior
- **Massively data-driven**
- Incorporates both qualitative and quantitative sources



8 Tips for Creating Agile Personas

1 Get to Know the Users

Research before creating

2 Keep Personas Concise

Don't overload with info

3 Distinguish User & Buyer

Different personas for each

4 Choose Primary Persona

Focus on main user type

5 Make Personas Believable

Add realistic details

6 Focus on Main Benefit/Problem

Core value proposition

7 Connect to User Stories

Link personas to requirements

8 Visualize Personas

Use images and graphics

Logical vs Physical Models



Logical Model

Analysis Phase

Shows WHAT the system must do, regardless of how it will be implemented physically

TOOLS:

DFD

Personas & User Stories

Class Diagram



Physical Model

Design Phase

Describes HOW the system will be constructed

TOOLS:

ERD

System Architecture

Network Architecture



Logical



Physical

Remember: Logical = **WHAT** | Physical = **HOW**

Lec 9 Data Flow Diagrams (DFD)

A DFD shows **how data moves** through an information system but **does NOT show program logic or processing steps**.

🔗 4 Basic DFD Symbols



Process

Transforms data



Data Flow

Movement of data



Data Store

Repository of data



Entity

External source/destination

Diagram 0 (Context Diagram)

Provides an overview of **ALL components** that interact to form the overall system.

⚠️ Process Problems to Avoid



Spontaneous Generation

Process produces output but has NO input data flow



Black Hole

Process has input but produces NO output



Gray Hole

Has input and output, but input is insufficient to generate output

📦 Leveling & Balancing

Leveling

Drawing a series of **increasingly detailed diagrams** until all functional primitives are identified.

Balancing

Maintains **consistency** by ensuring input and output data flows align properly.

↑↓ Higher-level = **Parent Diagram** | Lower-level = **Child Diagram**

Control Structures & Decision Tools

Data Element (data item/field) = The **smallest piece of data** that has meaning within an information system

Process Description

Documents the details of a **functional primitive** and represents specific processing steps and business logic.

Modular Design

Based on combinations of **3 logical structures** (control structures) as building blocks.

⚠ Each logical structure must have a **single entry and exit point**.

The 3 Control Structures



Sequence

Steps in sequential order, one after another



Selection

If statements - choose between paths



Iteration

Loop - repeat until condition met

Decision Tools



Decision Table

A logical structure that shows **every combination** of conditions and outcomes.

Decision Tree

A **graphical representation** of the conditions, actions, and rules found in a decision table.

CASE Repository

Ensures **data consistency**, especially important where multiple systems require the same data.

Lec 11

Object-Oriented Concepts



Object

Belongs to a group or category called a CLASS



Message

A command that tells an object to perform a certain method



Method

Defines specific tasks that an object can perform



Encapsulation

All data and methods are self-contained (black box concept)

Object Relationship Diagram

Can be prepared to provide an **overview of the system** and show how objects interact.



The Black Box Concept

An example of **encapsulation** - internal workings are hidden, only inputs/outputs are visible

Class vs Object

A **Class** is the blueprint/category, while an **Object** is an instance of that class.

Quick Reference

JAD

Joint Application Development - team-based fact-finding

RAD

Rapid Application Development - compressed full process

WBS

Work Breakdown Structure - breaking project into tasks

PERT

Program Evaluation Review Technique - complex project management

CPM

Critical Path Method - project scheduling

SRS

Software Requirements Specification - structured development doc

DFD

Data Flow Diagram - shows how data moves through system

FDD

Functional Decomposition Diagram - top-down process representation

UML

Unified Modeling Language - software systems design visualization

BPMN

Business Process Modeling Notation - requirements modeling standard

TOGAF

The Open Group Architecture Framework - aligns IT with business goals

Velocity

Story points completed per sprint - Scrum's central metric

Persona

Fictional character representing user types based on research

Encapsulation

All data and methods self-contained (black box concept)